

A Cost-Optimized Multi-Class Bayesian Agent for Cryptocurrency Rug-Pull Risk Assessment

A Sequential, Value-of-Information-Driven Approach to Token Risk Screening

[Author Name]

[Affiliation / Course: AI-Native AI Engineering]

1. Abstract

Assessing a cryptocurrency token for rug-pull risk normally means manually working through several unrelated sources of evidence — contract permissions, supply mechanics, fee logic, simulated trades, wallet flows, and social presence — before deciding whether to invest. This paper presents a probabilistic agent that treats this as sequential belief updating over seven mutually exclusive hidden states (one legitimate, five malicious, one inactive) via Bayes' theorem. Rather than running every check, the agent selects the next evidence category by Value of Information — Jensen–Shannon Divergence expected from each candidate test, normalized by its INR cost — so cheap, informative tests are preferred over expensive ones with similar expected gain. A cost-derived threshold (INR 5,000 false-positive cost vs. INR 10,000 false-negative cost) converts the evolving belief into PROCEED, BLOCK, or ESCALATE. Across 60 hand-built scenarios, the agent reached a verdict after checking an average of 2.44 of 6 categories, cutting evidence cost by 90.9% against an all-categories baseline, and matched the intended label on 87.1% of unambiguous cases. The same analysis surfaced a structural blind spot: the two most expensive, most diagnostic categories were never selected across any run, letting two scam scenarios clear as PROCEED on clean cheap-category evidence alone.

2. Introduction

Cryptocurrency “rug pulls” — schemes in which a token's creators drain value from investors after launch — are common enough that a large share of newly listed tokens turn out to be some form of scam rather than a functioning project. Establishing whether a specific token is safe to buy is not a single check; it is a sequence of them. A careful investor might read the smart contract for owner privileges, check whether the supply can be inflated, inspect the fee logic, simulate a buy and a sell to see whether the token can actually be exited, look at how concentrated the holdings are, and check whether the project has any real social presence — switching between tools and sources for each one. Doing this properly for every candidate token is slow, and skipping steps is exactly how obvious scams get missed.

Uncertainty is intrinsic to this problem, not incidental to it. No single piece of evidence proves a token is safe or malicious on its own — a renounced contract does not rule out a token-dump scheme, and a honeypot does not always reveal itself in its minting settings. Each test only shifts the odds. This is why the problem is naturally probabilistic: the right object to reason about is not “is this token a scam,” a binary the evidence usually cannot settle, but a distribution over the ways a token could be behaving, updated as each new test result comes in.

It is tempting to hand the whole judgment to a large language model — describe the token and ask whether it looks safe. Two problems make that insufficient here. First, the amount of capital at risk (this paper uses an illustrative INR 10,000 exposure) means a wrong answer has a real cost, and an LLM's free-text judgment does not come with a calibrated, auditable probability attached to it — there is no way to see how confident the model actually is or which piece of evidence moved that confidence. Second, an LLM has no principled way to decide which check to run next, or when it has gathered enough evidence to stop — it either answers from whatever context it is given or has to be told what to look at, which reintroduces the same evidence-selection problem this paper is trying to solve. A belief-based agent, by contrast, carries an

explicit probability over explicit hidden states, updates it with a transparent rule (Bayes' theorem), and can show its work: which category was checked, what it found, and how much that shifted the odds.

Given explicit beliefs, the next question is which evidence to gather and in what order. Not every check is equally useful, and not every check costs the same — in this system, reading static contract permissions is cheap (INR 10) while simulating an actual buy/sell round-trip is expensive (INR 150). Running every category regardless of what earlier evidence already implies wastes both money and time on tests unlikely to change the verdict. This is why the agent needs an explicit criterion for information selection: at each step it should run whichever remaining category is expected to move its beliefs the most per rupee spent, and stop as soon as its beliefs are decisive enough to act on. Cost is not an afterthought here — it is the reason a fixed checklist is the wrong design, and a greedy, budget-aware selection rule is the right one.

3. Related Work

This project did not start from academic literature on rug-pull detection or sequential evidence selection; the background reading was informal and aimed at understanding the domain rather than surveying prior formal approaches. It is recorded here honestly rather than backed with citations that were not actually used to shape the design.

The starting point was general orientation on how cryptocurrency and blockchain tokens work, via an overview published by PwC covering the basics of Bitcoin, blockchain, and cryptocurrency for a general audience. From there, the specific failure mode of interest — rug pulls — was identified by watching three explainer videos on YouTube covering how rug-pull scams are carried out in practice. These sources shaped the problem framing directly: the distinction between a liquidity-pool drain, a coordinated token dump, and a honeypot (buyers who cannot sell) as three different mechanisms rather than one generic “rug pull” category came from this video research, and is what motivated splitting the hidden-state space rather than using a single undifferentiated “scam” state.

No prior academic paper, dataset, or existing rug-pull-detection tool was consulted or reused. The Bayesian updating framework and the Value-of-Information selection criterion (Jensen–Shannon Divergence weighted by outcome probability, normalized by cost) were derived from general probability and information theory rather than adapted from a specific prior system. This is a limitation, not a strength — it means the hidden-state taxonomy and the likelihood tables in Section 4 reflect one person's reasoning about how these scams work rather than an evidence base drawn from real incident data, and it is flagged again in Section 12 (Limitations).

4. Probabilistic View

4.1 Hidden States and Their Origin

The agent's target is a distribution over mutually exclusive hidden states describing what a token actually is, not merely whether it is “good” or “bad.” This taxonomy went through five iterations. The first pass used three coarse states — genuine, rug pull, and some other malicious technique. Reading further revealed that “rug pull” is not one mechanism but several, so the second pass split it into the three most common variants: a liquidity-pool drain (the pooled trading capital behind the token is withdrawn by the insiders who control it), a token dump (insiders hold a large share of supply and sell it into the market once the price rises, collapsing it), and a honeypot (the contract allows buying but blocks selling). This produced five states — genuine, liquidity pull, share dumping, honeypot, and an “other malicious” catch-all. The catch-all state was then reframed as “Something Else” — covering dead, abandoned, or experimental projects rather than a vague malicious residual — both because a malicious catch-all had no coherent evidence

signature to update on, and because dead or abandoned tokens are a real, distinct category an investor can encounter.

A sixth and seventh state were added once it became clear the same evidence categories could distinguish further mechanisms without overlap: minting abuse (supply inflated after launch to dilute existing holders) and fee/tax rug pulls (extractive fee logic that drains value on every trade) are detectable from the same six evidence categories but are meaningfully different from a liquidity drain or a honeypot, so they were kept as separate states rather than folded into an existing one. This gave the final set of seven mutually exclusive states used throughout the system: Legitimate Project, Liquidity Rug Pull, Token Dump Rug Pull, Honeypot, Minting Abuse, Fee/Tax Rug Pull, and Something Else.

4.2 Priors and Their Origin

The prior distribution used at the start of every run is:

Hidden State	Prior
Legitimate Project	0.30
Liquidity Rug Pull	0.10
Token Dump Rug Pull	0.15
Honeypot	0.10
Minting Abuse	0.15
Fee/Tax Rug Pull	0.10
Something Else	0.10

These priors are hypothetical rather than fitted to data. Background reading turned up rough ranges for how common each failure mode is among newly listed tokens, but not point estimates precise enough to justify a fitted prior, so each value was set to a plausible midpoint of the range found rather than estimated from a dataset. This is an explicit simplification, not a claim of empirical grounding, and it is why the prior is treated as a fixed constant (Requirements, Section 1) rather than something the system could later learn or recompute.

4.3 Evidence Categories and Likelihoods

Evidence is partitioned into six independent categories so that no single underlying fact is counted twice. An earlier draft of the evidence set had eight categories, but two of them overlapped heavily with existing ones — running a near-duplicate check and folding its result back into the same Bayes update would inflate the belief for whichever state that overlapping evidence favored, double-counting one observation as if it were two independent ones. The six retained categories — Static Contract Permissions, Token Supply & Minting Mechanics, Fee & Tax Configuration, Runtime Execution Simulation, On-Chain Wallet Flows & Distribution, and Off-Chain & Social Viability — were kept because each observes a genuinely different aspect of the token: contract rules, supply mechanics, fee logic, simulated behavior, wallet-level distribution, and off-chain project activity, respectively. Each category yields exactly one of three mutually exclusive classifications (Section 3 of the technical specification), and each classification's likelihood under every hidden state is given by a hand-authored 7×3 table (spec Section 8; implemented in `likelihoods.py`), so that $P(\text{evidence} \mid \text{state})$ sums to 1.0 across each state's row.

4.4 A Full Worked Update

To make the update mechanism concrete, consider one evidence category observed against the prior above: On-Chain Wallet Flows & Distribution returns the outcome “Distributed.” The likelihood of this outcome under each hidden state (from the table in spec Section 8.5) is:

Hidden State	P(Distributed State)
Legitimate Project	0.82
Liquidity Rug Pull	0.05
Token Dump Rug Pull	0.02
Honeypot	0.20
Minting Abuse	0.30
Fee/Tax Rug Pull	0.40
Something Else	0.45

Bayes' theorem requires the unnormalized product of prior and likelihood for each state, then normalizing so the seven values sum to 1:

Hidden State	Prior × Likelihood	Posterior (normalized)
Legitimate Project	$0.30 \times 0.82 = 0.246$	61.0%
Liquidity Rug Pull	$0.10 \times 0.05 = 0.005$	1.2%
Token Dump Rug Pull	$0.15 \times 0.02 = 0.003$	0.7%
Honeypot	$0.10 \times 0.20 = 0.020$	5.0%
Minting Abuse	$0.15 \times 0.30 = 0.045$	11.1%
Fee/Tax Rug Pull	$0.10 \times 0.40 = 0.040$	9.9%
Something Else	$0.10 \times 0.45 = 0.045$	11.1%

The unnormalized column sums to 0.404; dividing each entry by this sum gives the posterior in the third column. A single “Distributed” wallet-flow result moves Legitimate Project from a 30% prior to a 61% posterior — past the halfway point after only one category — while leaving the two dump-oriented states (Liquidity Rug Pull, Token Dump Rug Pull) suppressed near 1%, since a coordinated dump is the least consistent explanation for widely distributed holdings. This is the same update mechanism (Bayes' theorem, spec Section 5) the agent runs once per selected category, implemented as a single pure function in `bayes.py` that takes the current belief, the observed outcome, and the likelihood table, and returns the new belief — with no knowledge of cost, evidence files, or the decision policy.

4.5 The Decision Threshold

Converting a belief into an action requires a threshold, and the threshold used here is derived from the cost model rather than chosen arbitrarily. The system assumes an illustrative INR 10,000 exposure: the cost of a false negative (approving a token that turns out malicious) is the full INR 10,000 loss, while the cost of a false positive (blocking a token that was actually legitimate) is set at INR 5,000, representing a missed opportunity rather than a realized loss — concretely, a foregone 50% return on the same capital. The optimal threshold that minimizes expected cost is the classic asymmetric-cost Bayes threshold:

$$T = \text{Cost}(\text{False Positive}) / [\text{Cost}(\text{False Positive}) + \text{Cost}(\text{False Negative})] = 5000 / (5000 + 10000) = 1/3 \approx 0.33$$

Because false negatives are twice as costly as false positives in this model, the threshold sits below the naive 0.5 midpoint — the agent should lean toward blocking sooner, since the cost of being wrong in the “approved a scam” direction is twice the cost of being wrong in the “blocked a good token” direction. Section 6 covers how this threshold is actually applied to the seven-state belief during and after the evidence loop, including a resolution of an inconsistency between the original technical specification and the implementation requirements that govern the code in this paper.

5. Information-Theoretic View

The only information-theoretic concept used in this system is Jensen–Shannon Divergence (JSD), used to measure how much a candidate piece of evidence is expected to move the belief distribution. No other information-theoretic machinery (e.g. mutual information, KL divergence directly, channel capacity) is used, in keeping with using only the concepts the system actually needed rather than adding measures for their own sake.

5.1 Jensen–Shannon Divergence

JSD is a symmetric, bounded measure of how different two probability distributions are. Given the current belief P (before a piece of evidence) and a candidate posterior Q (the belief the agent would have after observing one specific outcome), JSD is computed as the average of each distribution's KL divergence from their midpoint distribution $M = (P + Q) / 2$:

$$D_{JS}(P, Q) = \frac{1}{2} \cdot D_{KL}(P \parallel M) + \frac{1}{2} \cdot D_{KL}(Q \parallel M)$$

Unlike raw KL divergence, JSD is symmetric ($D_{JS}(P, Q) = D_{JS}(Q, P)$) and always finite even when P and Q assign zero probability to different states, which makes it well-suited to comparing a belief distribution before and after a hypothetical update without needing to guard against divide-by-zero or undefined-log cases that plain KL divergence can produce. In the worked example from Section 4.4, observing “Static Contract Permissions → Weaponized” against the fixed prior yields a JSD of approximately 0.2294 bits — a substantial, but not maximal, shift, consistent with “Weaponized” being strongly associated with Honeypot (spec Section 8.1: $P = 0.80$) without being exclusive to it.

5.2 Expected Information Gain

JSD alone measures the shift from one specific observed outcome, but before a category is actually run, its real outcome is unknown — only its three possible outcomes and their probabilities are. Expected Information Gain (EIG) resolves this by simulating all three possible outcomes for a candidate category, computing the hypothetical posterior and its JSD from the current belief for each one, and then weighting each outcome's JSD by that outcome's own marginal probability under the current belief (summed over all seven hidden states) before summing across the three outcomes. This gives a single expected value: how much this category is likely to move the belief, averaged over what it might actually report back, without ever looking at the real evidence file to find out which outcome will actually occur — a structural requirement (Requirements, Section 2.2) enforced by keeping this simulation logic (`voi.py`, `entropy.py`) entirely separate from the one module allowed to read real evidence (`evidence_input.py`).

6. Information Selection

Every remaining evidence category costs a different amount in INR (spec Section 7.1), and cost is not proportional to how informative a category tends to be — the cheapest categories are not necessarily the least useful, and the most expensive one is not guaranteed to resolve the case. The agent must therefore weigh information value against cost rather than picking the cheapest or the most informative category outright.

Evidence Category	Cost (INR)	What It Observes
Static Contract Permissions	10	Whether the owner retains and exercises privileged control
Token Supply & Minting Mechanics	10	Whether supply is capped or can be inflated post-launch
Fee & Tax Configuration	10	Whether trade fees are fixed, adjustable, or extractive
Off-Chain & Social Viability	40	Whether the project shows real, ongoing activity
On-Chain Wallet Flows & Distribution	80	How concentrated holdings are, and whether dumping is underway
Runtime Execution Simulation	150	Whether a simulated buy and sell actually succeed

The three cheapest categories (contract permissions, supply mechanics, fee configuration — INR 10 each) read static properties of the contract and are correspondingly cheap to check. Off-chain social viability sits in the middle at INR 40. The two most expensive categories are also the two that require the most active work: on-chain wallet flow analysis (INR 80) requires tracing transaction history across many wallets, and runtime execution simulation (INR 150) requires actually simulating trade execution rather than reading static state, making it the single most expensive — and, per the likelihood tables (spec Section 8.4), often the single most decisive — category, since a Honeypot's defining behavior (sell blocked) shows up there most directly ($P(\text{Execution-Reverted} \mid \text{Honeypot}) = 0.95$).

At each step, the agent computes Value per Rupee for every remaining category — its Expected Information Gain (Section 5.2) divided by its INR cost — and greedily selects whichever remaining category has the highest ratio (spec Section 4, steps 1–6; implemented in `voi.py`). This is why the agent does not simply run categories in a fixed order, or cheapest-first, or most-expensive-first: a category with modest expected information gain but very low cost can outrank a category with higher absolute expected gain but much higher cost, and the ranking itself changes step to step as the belief distribution updates — a category that looked most valuable against the prior may no longer be the best choice once one piece of real evidence has already narrowed the distribution. The agent picks what it picks because, at that specific point in the loop, no other remaining category is expected to move the belief further per rupee spent.

7. Decision Policy

The technical specification's original decision policy (Sections 6.3 and 7.2) applied one shared threshold ($\text{Risk Score} \geq 0.33$, i.e. the five malicious states summed) at two separate checkpoints — mid-loop and end-of-loop. Working through this rule during implementation surfaced a structural problem, documented directly in the specification itself (Section 9, Open Items to Reconcile): if the mid-loop check never fires across every category, the Risk Score has necessarily stayed under 0.33 the entire run with no evidence left to change it, so the end-of-loop check — using the identical threshold — can only ever land on PROCEED-equivalent territory. Its own BLOCK branch, and by extension the whole ESCALATE branch, would then be unreachable by construction, not merely rare in practice. The implementation requirements resolve this by replacing the combined-threshold rule entirely with two distinct checks, given in full below; the spec's Sections 6.3 and 7.2 are not implemented as written.

7.1 Action Set

The agent's output is exactly one of four verdicts: CONTINUE (an internal loop-control state, never a final answer — keep gathering evidence), PROCEED (approve the token), BLOCK (reject the token), or ESCALATE (neither threshold was met with all evidence exhausted; refer for manual review).

7.2 Derived Threshold

The 0.33 / 0.67 pair used throughout the mid-loop check (below) is the same cost-derived threshold from Section 4.5 ($T \approx 1/3$), applied in two directions: 0.33 as the bar for a single malicious (or inactive) state to justify blocking, and its complement 0.67 as the bar for Legitimate Project alone to justify proceeding. The two are symmetric around the same threshold, not independently chosen numbers.

7.3 Mid-Loop Check (after every real evidence update)

Inspects each of the seven states' own individual probability — never a summed score:

- If Legitimate Project's probability alone is $\geq 0.67 \rightarrow$ PROCEED.
- Else if any one of the five malicious states (Liquidity Rug Pull, Token Dump Rug Pull, Honeypot, Minting Abuse, Fee/Tax Rug Pull) individually has probability $\geq 0.33 \rightarrow$ BLOCK.
- Else if Something Else's probability alone is $\geq 0.33 \rightarrow$ BLOCK.
- Else $\rightarrow$ CONTINUE (loop again if categories remain).

PROCEED and BLOCK deliberately use different bars (0.67 vs. 0.33) because they are not symmetric claims: PROCEED requires a single state (Legitimate Project) to dominate the entire distribution, while BLOCK only requires one malicious explanation to clear the cost-derived risk threshold, regardless of how the remaining probability is spread across the other states. No state in the fixed prior (Section 4.2) reaches either bar on its own, so this check cannot fire before at least one real category has been checked.

7.4 End-of-Loop Check (only if every category is checked and Section 7.3 never fired)

- Sum the five malicious states plus Something Else.
- If that sum is $> 0.67 \rightarrow$ BLOCK.
- Else $\rightarrow$ ESCALATE.

This is the only point in the whole decision policy where a combined, summed score is used; every mid-loop check inspects individual states only. ESCALATE means the agent has exhausted all six evidence categories, remains under the block threshold, but has not accumulated enough belief in Legitimate Project specifically to close the case out as PROCEED on its own — a genuinely ambiguous result that is routed to manual review rather than forced into an automatic verdict. What that manual review process actually consists of is out of scope for this prototype (Section 12).

7.5 Expected Cost and Stopping

Because the agent stops as soon as either mid-loop bar is crossed, the total INR spent on evidence in a given run is the sum of only the categories actually checked before that point — not a fixed cost per run. A token whose very first, cheapest check (Static Contract Permissions, INR 10) already produces a decisive result never triggers the expensive runtime-simulation check (INR 150) at all. This is the practical payoff of Sections 5–6: cost is not spent uniformly across scenarios, but concentrated on the cases that actually need more evidence to resolve. Section 8 reports how this plays out quantitatively across the seeded scenarios.

8. Experiment

Two batches of scenario files were constructed and run through the completed agent (`main.py`, together with `bayes.py`, `voi.py`, `entropy.py`, `decision.py`, `evidence.py`, `states.py`, and `evidence_input.py`) to test different things, using the fixed prior (Section 4.2) reset at the start of every run.

8.1 Batch 1 — Scenario Diversity (50 scenarios)

Fifty hand-constructed scenario files, each specifying one pretend token's true outcome for all six evidence categories (Requirements, Section 2), were built to exercise the agent across a spread of cases rather than just the three examples in the requirements appendix. They fall into six informal groups, identifiable by filename prefix: `legit` (10 files, each varying which category shows a slightly less-than-ideal but still legitimate-consistent outcome, e.g. `Centralized-Idle` instead of `Renounced`), `honeypot` (10 files, varying how the honeypot behavior surfaces), and one file each for `liquidity rug`, `token dump`, `minting abuse`, and `fee/tax rug` as headline cases, plus 10 mixed-signal files (clean-looking evidence in most categories with one category deliberately contradicting the rest), 2 dead/experimental-project files, and 8 edge-case files probing single- and multi-“yellow-flag” situations, including one all-worst-outcomes and one all-middle-outcomes stress test. No randomness is involved anywhere in this pipeline — the Bayesian update (`bayes.py`) and the Value-of-Information selection (`voi.py`) are both deterministic functions of the current belief and the fixed likelihood tables, so no seed is needed; every outcome in every scenario file was chosen by hand, not sampled.

Where a scenario's filename indicates an unambiguous intended state (`legit`, `honeypot`, `liq_rug`, `token_dump`, `mint_abuse`, `tax_rug` — 31 of the 50 files), the expected verdict was taken to be `PROCEED` for `legit` and `BLOCK` for the five malicious groups, and the agent's actual `final_answer` was checked against it. The remaining 19 files (mixed, dead, and edge-case groups) were deliberately built to be ambiguous or to stress specific decision-boundary behavior and were not scored against a single “correct” label, consistent with how `scenario_mixed_signals.json` is treated in the requirements (Section 2.3): what the agent actually produces on these is itself the result of interest, not a pass/fail check.

8.2 Batch 2 — Decision-Policy Checkpoints (10 scenarios)

A second, smaller batch of ten scenario files was built specifically to try to exercise each named branch of the decision policy (Section 7) at least once: the `PROCEED` branch, a `BLOCK` triggered by each of the five malicious states individually, a `BLOCK` triggered by `Something Else`, the `PROCEED-takes-precedence-over-BLOCK` ordering, and the two end-of-loop outcomes (`BLOCK` and `ESCALATE`). Each file's outcomes were chosen by hand to try to steer the belief toward the named target path. Section 9.3 reports how many of these ten actually landed on the path their filename intended.

8.3 Baseline for Comparison

To measure what the Value-of-Information selection and early-stopping rule (Sections 6–7) actually save, the total INR cost the agent spent across the 50 Batch 1 scenarios was compared against a fixed baseline: the cost of checking all six evidence categories on every single scenario regardless of the mid-loop check, i.e. $\text{INR } 10 + 10 + 10 + 40 + 80 + 150 = \text{INR } 300$ per scenario (spec Section 7.1), multiplied by 50 scenarios.

8.4 Metrics

For every run: the `final_answer` (`PROCEED` / `BLOCK` / `ESCALATE`), the `confidence_score`, the number of categories actually checked before stopping (of a possible 6), the `total_cost_inr`, and — for Batch 1's 31 unambiguous scenarios — whether `final_answer` matched the label implied by the scenario's filename. All figures reported in Section 9 are read directly from the result JSON files produced by `main.py`; none are estimated.

9. Results

9.1 Overall Accuracy and Cost (Batch 1, 50 scenarios)

Across all 50 Batch 1 scenarios, the agent reached a verdict after checking an average of 2.44 of the 6 categories (median 3, range 1–4), at an average cost of INR 27.4 per scenario (median INR 30). No run ever reached ESCALATE or checked all 6 categories. On the 31 scenarios with an unambiguous filename-implied label, the agent's final_answer matched that label in 27 cases (87.1% agreement); the 4 disagreements are examined in Section 10. Verdicts split 17 PROCEED / 33 BLOCK across all 50 scenarios (including the deliberately ambiguous mixed/dead/edge groups, which were not scored).

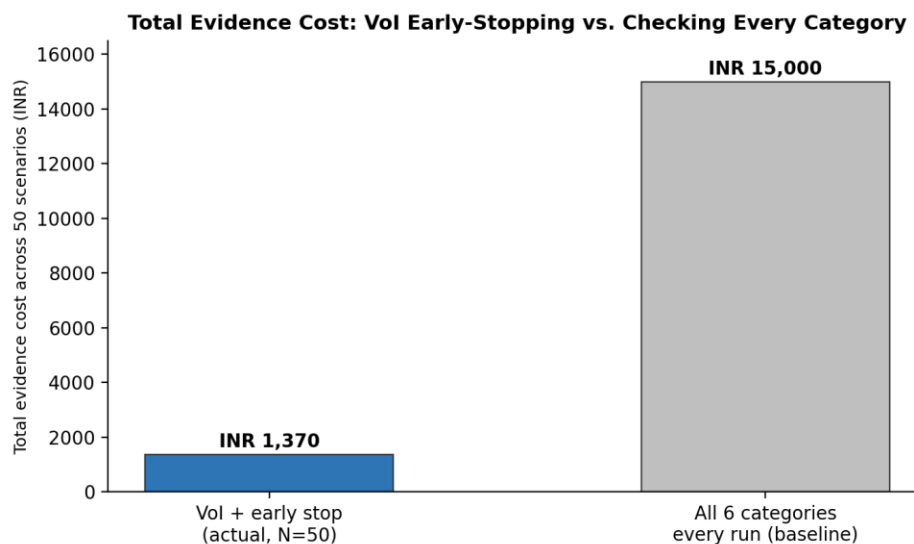


Figure 1. Total evidence cost summed across the 50 Batch 1 scenarios: the agent as run (Value-per-Rupee selection with mid-loop early stopping) versus a baseline that checks all six categories on every scenario regardless of belief.

The as-run total across all 50 scenarios was INR 1,370, against an all-categories baseline of INR 15,000 — a 90.9% reduction in evidence cost for this scenario set. This is the direct, measured payoff of Sections 6–7: the agent is not spending a fixed amount per token, and in this batch it never needed to.

9.2 Which Categories Actually Get Selected

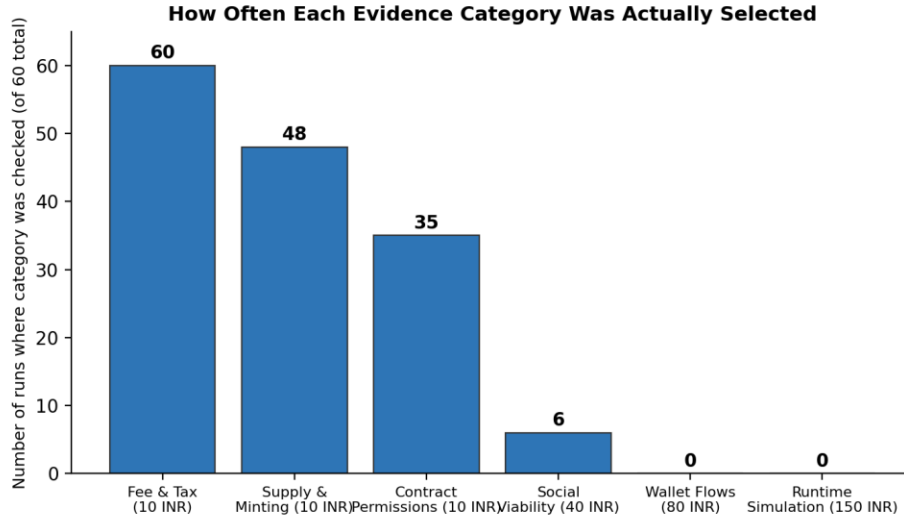


Figure 2. Number of runs (out of all 60 scenarios across both batches) in which each evidence category was actually checked.

Fee & Tax Configuration was selected first in every single run (60/60), and was checked at all in every run. Token Supply & Minting Mechanics and Static Contract Permissions were checked often (48 and 35 times respectively) as the loop continued past the first category. Off-Chain & Social Viability was checked only 6 times, and — across all 60 scenarios in both batches — On-Chain Wallet Flows & Distribution and Runtime Execution Simulation were never checked, not once. This was verified independently: recomputing Expected Information Gain and Value per Rupee by hand against the fixed prior (Section 4.2) and the published likelihood tables (spec Section 8) confirms Fee & Tax Configuration does have the highest value-per-rupee at the very first step (0.0124), ahead of Token Supply & Minting (0.0114) and Static Contract Permissions (0.0105) — and that Runtime Execution Simulation, despite having the single highest raw expected information gain of any category (0.130 bits, versus 0.124 for Fee & Tax), has the lowest value-per-rupee (0.00086) because its INR 150 cost is 15× that of the cheap categories. This is not an implementation bug; it is the greedy Value-per-Rupee rule (Section 6) working exactly as specified — but it means that, at least against this prior and this likelihood table set, the two most expensive categories are structurally very unlikely to ever be selected, since three cheap categories are almost always available and almost always offer a better ratio. Section 10 discusses what this costs the agent in the specific cases where it matters.

9.3 Decision-Policy Checkpoint Coverage (Batch 2, 10 scenarios)

Batch 2 was designed to exercise each named branch of the decision policy (Section 7) at least once. In practice, only 3 of the 10 files landed on the specific path their filename targeted:

File (intended path)	Actually stopped on	Verdict
policy_01_mid_loop_proceed	Legitimate Project $\geq 67\%$ (as intended)	PROCEED
policy_02_mid_loop_block_honeypot	Liquidity Rug Pull $\geq 33\%$ (not Honeygot)	BLOCK
policy_03_mid_loop_block_mint_abuse	Minting Abuse $\geq 33\%$ (as intended)	BLOCK
policy_04_mid_loop_block_fee_tax_rug	Honeypot $\geq 33\%$ (not Fee/Tax Rug)	BLOCK
policy_05_mid_loop_block_token_dump	Legitimate Project $\geq 67\%$ (not a BLOCK at all)	PROCEED

File (intended path)	Actually stopped on	Verdict
policy_06_mid_loop_block_liquidity_rug	Something Else $\geq 33\%$ (not Liquidity Rug)	BLOCK
policy_07_mid_loop_block_something_else	Legitimate Project $\geq 67\%$ (not a BLOCK at all)	PROCEED
policy_08_precedence_legit_first	Legitimate Project $\geq 67\%$ (as intended)	PROCEED
policy_09_end_of_loop_block	Mid-loop BLOCK after 2 categories (never reached end-of-loop)	BLOCK
policy_10_end_of_loop_escalate	Mid-loop BLOCK after 3 categories (never reached end-of-loop)	BLOCK

None of the 60 scenario files across both batches produced an ESCALATE verdict, and neither file built specifically to reach the end-of-loop check (policy_09, policy_10) actually got there — both resolved earlier via the mid-loop check instead. This does not mean the end-of-loop / ESCALATE code path is broken; it means that hand-picking six category outcomes that keep every individual state's posterior under both the 0.33 and 0.67 bars (Section 7.3) for all six categories at once is harder than it looks, precisely because the mid-loop check only needs one state to cross one bar to fire early. This is itself a finding, not just a batch-construction miss, and is carried into Section 10.

10. Failure Analysis

Five distinct failure patterns were identified from the 60 result files, classified by root cause rather than just listed as wrong answers.

Failure 1 — A honeypot cleared as PROCEED because the categories that would have caught it were never checked

test_12_honeypot_stealth_reverted (token StealthTrapToken) was built as a honeypot — its defining property is that Runtime Execution Simulation should return Execution-Reverted. But its three cheap categories (Fee & Tax: Low-Immutable, Supply: Hard-Capped, Contract: Renounced) all read as clean, and per Section 9.2 those are the only three categories the agent ever reaches for in practice. Legitimate Project crossed 67% after exactly those three checks, and the agent stopped and returned PROCEED at 71% confidence — without ever running the one check (Runtime Execution Simulation, $P(\text{Execution-Reverted} \mid \text{Honeypot}) = 0.95$ per spec Section 8.4) that would have caught it directly. Root cause: this is not a bad prior or a bad likelihood table entry — it is a structural interaction between the greedy Value-per-Rupee rule (Section 6) and a scenario designed so that every cheap, frequently-selected category happens to look clean. Nothing was changed in response within this prototype (Requirements, Section 6 scopes this out), but it is flagged again in Section 13's new questions.

Failure 2 — The same blind spot catches a token-dump scenario

test_22_token_dump_insider (InsiderDumpExpress) shows the identical pattern: built as a token-dump rug pull, whose signature is a Rug-Dumping wallet-flow pattern (On-Chain Wallet Flows & Distribution), but the same three cheap categories again read clean, Legitimate Project again crossed 67% after 3 checks, and the wallet-flow category — the one category that would have surfaced this specific mechanism — was never reached. Root cause: identical to Failure 1. Grouping these as one pattern rather than two separate failures matters for Section 13's framing: this is a systemic property of the current prior/likelihood/cost combination, not two unrelated coincidences.

Failure 3 — A legitimate-labeled token blocked on weak, borderline evidence

test_10_legit_corporate_web3 (EnterpriseChain) was built as a legitimate project, but was BLOCKed after only 3 cheap categories at 33.2% confidence — just barely over the 0.33 bar (Section 7.3), on a token-dump reading driven by an Adjustable-Moderate fee outcome and a Centralized-Idle permissions outcome. test_08_legit_dao_utility (GovernanceUtility) shows the same shape: BLOCK at 34.7% on similarly moderate (not extreme) category outcomes. Root cause: the mid-loop BLOCK bar (0.33) can fire on a belief that is barely past the threshold and still has over 65% of its mass elsewhere — the check asks only whether one malicious state individually clears 0.33, not how much stronger it is than the runner-up. A token with several moderately-suspicious (not clearly malicious) cheap-category outcomes can cross that bar on accumulated ambiguity rather than strong evidence, and stops before the more decisive categories are ever reached.

Failure 4 — The end-of-loop / ESCALATE path could not be reached even when specifically targeted

policy_09_end_of_loop_block and policy_10_end_of_loop_escalate (Section 9.3) were both hand-built specifically to reach the end-of-loop check (Section 7.4), and both instead resolved via the mid-loop check after only 2–3 categories. Root cause: constructing six category outcomes that keep every one of the seven states' individual posteriors under both the 0.33 and 0.67 bars simultaneously, across all six updates, is a much narrower target than it appears — the mid-loop check only needs one state to cross one bar once to fire, so avoiding that for the full loop length is difficult by hand. Combined with zero ESCALATE verdicts across all 60 files (Section 9.3), this suggests the end-of-loop / ESCALATE branch may be difficult to reach in practice with this prior and these likelihood tables, not just difficult to hit on purpose — echoing, in a different form, the reachability concern the original specification itself raised about its own (superseded) threshold rule (spec Section 9, Open Item 2).

Failure 5 — Two policy-checkpoint files blocked on a different state than intended

policy_02_mid_loop_block_honeypot was built to demonstrate a Honeypot-triggered BLOCK, but actually stopped on Liquidity Rug Pull reaching 41.4%. policy_04_mid_loop_block_fee_tax_rug was built to demonstrate a Fee/Tax-Rug-triggered BLOCK, but actually stopped on Honeypot reaching 36.8%. Root cause: several of the seven hidden states share overlapping likelihood signatures on the same three cheap, commonly-checked categories (Section 9.2) — a fee or permissions outcome that is highly suggestive of one malicious state is often also somewhat suggestive of another, so whichever state's posterior happens to cross 0.33 first is sensitive to the exact prior/likelihood numbers rather than cleanly separable by category outcome alone. Both runs still correctly returned BLOCK — the practical verdict was right — but the specific malicious state identified was not the one the scenario intended to demonstrate, which matters if the identified state is ever surfaced to a user as “why” the token was blocked.

Taken together, Failures 1–2 are the most consequential for real use: they show the current cost structure and prior create a small but real blind spot where a scam token can pass simply by having clean-looking cheap-category evidence, since the expensive categories best suited to catching it are, per Section 9.2, essentially never reached under the greedy rule as currently weighted.

11. Human Discussions

No discussion with another person took place during this project. The idea came together suddenly while already partway into a different, unrelated project, and the work from initial design through implementation and this write-up was done solo and continuously without pausing to get feedback from a classmate, mentor, or TA. This is stated plainly rather than invented, per the same honesty standard this paper applies to Related Work (Section 3).

12. Limitations

Several limitations are worth stating plainly rather than glossing over.

1. Priors and likelihoods are hypothetical, not fitted. As noted in Section 4.2, the prior distribution and all 42 entries across the six likelihood tables (Section 4.3) were authored by hand from general reasoning about how each scam mechanism tends to present, informed only by the limited background reading described in Section 3 — not fitted to a labeled dataset of real rug pulls. The system's outputs are only as trustworthy as these hand-set numbers, and different reasonable numbers would shift every result in this paper.
2. Evidence categories return a simulated, pre-decided outcome. Per the requirements (Section 2), there is no real blockchain connection, no real contract parser, and no real social-media check behind any of the six evidence categories — each scenario's true outcome is simply written into a JSON file by the developer ahead of time. The Bayesian update mechanism, the VoI selection logic, and the decision policy are all validated against these pretend outcomes, not against real tokens.
3. No real money or transactions are involved. The INR 10,000 / INR 5,000 cost model (Section 4.5) and the per-category evidence costs (Section 6) are bookkeeping figures used to derive a threshold and rank categories — nothing is actually spent, staked, or transacted.
4. The ESCALATE path is a dead end in this prototype. As stated in the specification itself (spec Section 9) and left explicitly out of scope in the requirements (Section 6), what happens to a token that reaches ESCALATE — who reviews it, on what timeframe, what happens to the trade meanwhile — is entirely undefined here.
5. The hidden-state taxonomy may not be exhaustive. Section 4.1 describes five rounds of revision to arrive at seven states, but there is no guarantee real rug pulls fall cleanly into exactly these seven mutually exclusive categories rather than combining mechanisms (e.g. a token that is both a honeypot and a minting-abuse scheme simultaneously) that this model cannot represent.
6. Related Work (Section 3) is thin. The design was not checked against any existing academic or industry approach to rug-pull detection, so there is no way to say from this paper alone whether the chosen states, categories, or thresholds are better, worse, or simply different from established practice.

13. New Questions

Mandatory, per the rubric — the questions this work raised rather than answered:

1. How sensitive is the final verdict to the hand-set likelihood table values? A systematic perturbation study — nudging individual likelihood entries and re-running the seeded scenarios — would show whether the decision policy is robust or whether a handful of specific numbers are quietly load-bearing.
2. Would a learned or fitted prior and likelihood table, trained on a real dataset of confirmed rug pulls and legitimate tokens, change which categories the VoI selection tends to prefer — and specifically, would Runtime Execution Simulation and On-Chain Wallet Flows ever win the value-per-rupee ranking, given that across all 60 scenarios run in this paper (Section 9.2) they never did even once?
3. Should the greedy selection rule be modified so that certain categories (e.g. Runtime Execution Simulation) are guaranteed to run at least once before a PROCEED verdict is allowed, given that Failures 1–2 (Section 10) show a token can clear entirely on cheap-category evidence without ever being tested for the specific failure mode — sell-blocking, coordinated dumping — its category taxonomy was built to catch?

4. What should the ESCALATE path actually route to in a real deployment — a second, more expensive automated pass, a human reviewer with a time budget, or a default-to-block policy under time pressure — and how would each choice change the effective cost model in Section 4.5?
5. Does the greedy, one-step-lookahead Value-of-Information rule (Section 6) ever underperform a rule that looks two steps ahead — i.e. are there cases where the single best next category is not part of the best pair of categories to check together?
6. How would this framework need to change to handle evidence that arrives already contradictory or noisy — for example, a runtime simulation that succeeds on one node's RPC endpoint and fails on another — rather than the single, clean outcome per category assumed throughout this prototype?

AI-Use Statement

AI assistance (Claude, Anthropic) was used throughout this project in two distinct ways. First, for research and understanding: Claude was used to learn how blockchain and cryptocurrency mechanics work at a level beyond the introductory PwC article and YouTube videos in Section 3, to work through how each of the six evidence categories actually maps onto detectable on-chain and off-chain signals, and to explore and narrow down candidate hidden-state taxonomies during the five revisions described in Section 4.1. Second, for implementation and this write-up: Claude was used to help write and review the Python implementation (`main.py`, `likelihoods.py`, and the other modules referenced throughout this paper) against the technical specification and requirements documents, and to draft and edit the prose of this paper itself from the author's own notes, code, and the raw result JSON files.

AI-generated or AI-assisted parts: the Python code implementing the Bayesian update, Value-of-Information selection, decision policy, and entropy calculations; the structure and prose of this paper (Sections 1–13, this statement, and the appendix); and the two charts in Section 9, generated from the author's own result data. The hidden-state taxonomy, evidence categories, prior values, likelihood table numbers, cost model, and decision thresholds are the author's own design choices (Sections 4–7), refined through discussion with Claude but not generated by it wholesale.

Verification: every numeric claim in Sections 8–10 was checked against the actual result JSON files the author generated by running the completed agent — accuracy, cost totals, category-selection counts, and the specific per-scenario numbers were computed directly from those files (not estimated), and the Value-per-Rupee ranking reported in Section 9.2 was independently re-derived by hand from the fixed prior and the published likelihood tables to confirm it was a genuine property of the math rather than an implementation bug.

Experiments personally run: the author ran `main.py` against all 60 scenario files (the 50-scenario diversity batch and the 10-scenario decision-policy batch described in Section 8) and supplied the resulting result JSON files used throughout Sections 9–10. All scenario input files — the specific evidence outcomes assigned to each pretend token — were also constructed by the author by hand.

Appendix A: Twenty Core Questions

Answered in the author's own words, using this paper's own system wherever possible, per the course's mandatory question set.

1. What is a hidden state, and what are yours?

A hidden state is one member of a set of possible underlying realities that cannot be observed directly, only inferred from evidence. This paper's seven hidden states (Section 4.1) are Legitimate Project, Liquidity Rug

Pull, Token Dump Rug Pull, Honeypot, Minting Abuse, Fee/Tax Rug Pull, and Something Else — exactly one of which is true of a given token at any time, though the agent never gets to see which.

2. Why is directly predicting an answer sometimes insufficient?

A direct prediction collapses everything the system knows into one label with no visible confidence or reasoning attached, which is risky when a wrong answer has a real cost (Section 4.5's INR 10,000 exposure) and there is no way to audit how the answer was reached. This project uses a full belief distribution instead of a single prediction precisely so a human can see how much the evidence actually supports the verdict, and which specific piece of evidence moved it.

3. What is a belief distribution, and why must it sum to one?

A belief distribution is the full set of probabilities assigned to every hidden state at once — in this system, the seven numbers in `final_belief`. It must sum to one because the seven states are mutually exclusive and collectively exhaustive by construction (Section 4.1): the token must be exactly one of them, so the probabilities of all the ways it could be are the total certainty available, no more and no less.

4. What is the difference between prior and posterior?

The prior is the belief distribution before a specific piece of evidence is considered — in this system, the fixed starting distribution in Section 4.2 before any category has been checked. The posterior is the belief distribution after that evidence has been folded in via Bayes' theorem (Section 4.4); the posterior from one update becomes the prior for the next.

5. What is likelihood, and how does it differ from probability?

Likelihood is $P(\text{evidence} \mid \text{hidden state})$ — how probable a particular observed outcome is, assuming one specific hidden state is true — whereas the belief itself, $P(\text{hidden state} \mid \text{evidence})$, is a genuine probability distribution over states that sums to one. The six likelihood tables (spec Section 8) hold $P(\text{outcome} \mid \text{state})$ values, and each state's row sums to one across its outcomes, but the likelihood numbers across different states for the same outcome do not sum to anything meaningful — they answer “how expected is this outcome if this were true,” not “how probable is this outcome.”

6. What does it mean for an agent to be uncertain?

An agent is uncertain when its belief distribution has meaningful probability mass spread across more than one hidden state, rather than concentrated almost entirely on one. Concretely, the fixed prior (Section 4.2) starts uncertain by design — no single state above 30% — and the mid-loop check (Section 7.3) is exactly a test of whether that uncertainty has resolved enough (one state past 0.67 or 0.33) to act on.

7. What is entropy, and why does it represent uncertainty?

Entropy is a single number summarizing how spread out a probability distribution is, computed as $-\sum P(\text{state}) \cdot \log_2 P(\text{state})$ across all states (`entropy.py`). It represents uncertainty because it is at its maximum when belief is spread evenly across all seven states (nothing is favored) and at its minimum, zero, when one state holds all the probability (nothing is left to learn).

8. What is a bit, in plain words?

A bit is the amount of information needed to cut a set of equally likely possibilities exactly in half — one yes/no question that is equally likely to go either way. When this paper reports the worked example's JS divergence as 0.2294 bits (Section 5.1), that is a fraction of one such halving worth of belief-shift, not a literal binary digit.

9. What is information gain, and how does it relate to entropy?

Information gain is the reduction in entropy — or, in this system, the Jensen–Shannon Divergence — produced by observing a piece of evidence, measuring how much less spread-out (or how differently shaped) the belief becomes after updating. Expected Information Gain (Section 5.2) is exactly this quantity

averaged over a category's three possible outcomes, weighted by how likely each outcome is, before that category is actually run.

10. What is conditional entropy?

Conditional entropy is the expected remaining entropy of a distribution after conditioning on a piece of evidence, averaged over that evidence's own possible outcomes. `entropy.py`'s `expected_entropy_if_run` function computes exactly this for each unchecked category — the entropy the belief is expected to have left over if that category were run next — reported in the result JSON's `unchecked_categories` field (Requirements, Section 5).

11. What is mutual information, and how does it differ from correlation?

Mutual information measures how much knowing one variable reduces uncertainty about another, in any relationship — not just a linear one — whereas correlation only captures straight-line association and can be zero even when two variables are strongly, non-linearly related. In this system, the expected reduction in entropy from checking a category (Question 10) is this project's stand-in for mutual information between that category's outcome and the hidden state, even though the categories' likelihood tables encode relationships that are not simple linear correlations.

12. What is KL divergence, and why is it not a distance?

KL divergence $D_{KL}(P \parallel Q)$ measures how much information is lost when approximating one distribution P with another Q , computed as $\sum P(s) \cdot \log_2(P(s)/Q(s))$. It is not a true distance because it is not symmetric (Question 13) and does not satisfy the triangle inequality, so “how far Q is from P ” and “how far P is from Q ” are, in general, different numbers.

13. Why is KL divergence asymmetric? Give a small example.

KL divergence is asymmetric because it specifically measures surprise relative to a reference distribution, and swapping which distribution is the reference changes what counts as surprising. For a small example inside this project: this system's prior assigns Minting Abuse 0.15 and Honeypot 0.10; treating the prior as reference ($D_{KL}(\text{prior} \parallel \text{uniform})$) penalizes the prior's departure from an even 1/7 spread, while treating a hypothetical posterior that has collapsed Minting Abuse to near 1.0 as reference ($D_{KL}(\text{posterior} \parallel \text{prior})$) heavily penalizes the prior for having assigned that now-certain state only 15% — the same two distributions, but a very different divergence value depending on which one is being measured against the other.

14. What is Jensen–Shannon divergence, and why was it introduced?

Jensen–Shannon Divergence (Section 5.1) is a symmetrized, bounded version of KL divergence, computed as the average of each distribution's KL divergence from their midpoint. It was introduced here specifically because the Value-of-Information step (Section 6) needs to compare a prior belief to a hypothetical posterior without worrying about which one is the “reference” distribution or about undefined values when one distribution assigns zero probability somewhere the other does not — both problems that plain KL divergence has.

15. What is calibration, and why does it matter for an agent?

Calibration means that when an agent reports a given confidence level, that confidence should match the actual real-world frequency of being correct at that level — a token flagged PROCEED at 71% confidence should, across many tokens, turn out legitimate roughly 71% of the time. It matters here because `confidence_score` is what a person is meant to act on directly; Section 10's Failures 1–3 are, among other things, examples of the system reporting a specific confidence number without any real dataset to check whether that number is actually calibrated to real outcomes.

16. What is expected cost, and how did you derive your threshold from it?

Expected cost is the probability-weighted average financial cost of a decision, combining the cost of each possible error with the probability of making it. Section 4.5 derives the 0.33 threshold by minimizing expected cost under two asymmetric error costs — INR 10,000 for a false negative (approving a scam) and INR 5,000 for a false positive (blocking a legitimate token) — giving $T = 5000 / (5000 + 10000) = 1/3$, the risk level at which blocking and proceeding cost the same in expectation.

17. What is value of information? When is more information not worth it?

Value of information is how much a piece of evidence is expected to improve a decision, weighed against what that evidence costs to obtain — operationalized in Section 6 as Expected Information Gain divided by INR cost. More information is not worth it once the belief has already crossed a decision threshold (Section 7.3's mid-loop check firing) or when every remaining category's value-per-rupee is low enough that its expected shift in belief doesn't justify its cost — which, per Section 9.2's finding, turned out to be true of Runtime Execution Simulation and On-Chain Wallet Flows in every single one of the 60 runs tested.

18. Can a question be highly informative and still useless? Give your own example.

Yes — a question can shift beliefs a great deal on average while still not being worth asking if it costs too much relative to that shift, or if cheaper questions already achieve nearly the same effect. Runtime Execution Simulation is exactly this case in Section 9.2: it has the single highest raw expected information gain of any category (0.130 bits) yet the lowest value-per-rupee (0.00086) because its INR 150 cost outweighs that gain — informative in the absolute sense, but not worth its price given cheaper alternatives.

19. What is distribution shift, and how would your agent detect it?

Distribution shift is when the real-world relationship between evidence and outcomes drifts away from what a system's likelihood tables or priors assume — for example, if scammers began deliberately engineering clean-looking Fee & Tax and Supply outcomes specifically because they know those are the categories most agents like this one check first. This prototype has no built-in way to detect that on its own, since its likelihood tables are fixed (Section 4.3) and never updated from outcomes; noticing drift would currently require a human periodically comparing the agent's verdicts against real-world results, which is out of scope here (Section 12).

20. When should your agent stop searching and act? State the rule.

The agent stops and acts the moment either mid-loop bar is crossed after a real update: PROCEED if Legitimate Project alone reaches ≥ 0.67 , or BLOCK if any single malicious state or Something Else alone reaches ≥ 0.33 (Section 7.3) — and only if every category is exhausted without either firing does it fall through to the end-of-loop combined check (Section 7.4), which either BLOCKs (combined score > 0.67) or ESCALATES.

References

Only sources actually read (per the rubric instruction accompanying Related Work, Section 3):

PwC. “Bitcoin, Blockchain & Cryptocurrency: Fintech Basics.”

<https://www.pwc.com/us/en/industries/financial-services/fintech/bitcoin-blockchain-cryptocurrency.html>

Three YouTube videos on crypto rug pulls (titles and channel names not independently confirmed at time of writing — cited by URL only, per the course's rule against citing sources not directly read/verified; add titles and channel names directly from the videos before final submission):

<https://youtu.be/YFaqng3YESE>

<https://youtu.be/RwHGp4mELbU>

<https://youtu.be/dVJzcFDo498>